

Assignment 1

Name: Youssef Hossam Eldeen Mohamed Elzainy

ID: 212600

Part 1

Q1)

Because the circuits and the electronic devices is becoming more complicated day after another, so it became so hard to draw a full schematic manually.

That's why we began using HDLs, because it made it much easier to design a full schematic with a single block of code which is much simpler than drawing it manually.

Q2)

The main difference between the HDLs and software languages that HDLs is used for describing hardware (Hardware Description Language) While the software programming languages is used to give tasks and instructions for the CPU to execute one after another.

HDLs handles tasks in parallel (all blocks are handled in parallel) while the software programming languages handles the Code task by task not in parallel.

Q3)

The IEEE 1800 – 2009 Standard is for the SystemVerilog HDL which is (systemVerilog and Verilog combined together)

It was helpful because it allowed us to use the same language for both designing and verifying (Design and Verification)

Part 2

Q1)

Structural domain

Physical domain

Behavioral domain

Q2)

Because the Behavioral describes what the hardware does without specifying the actual hardware to be used while the structural level gives the synthesis tool all the info it needs to know about what hardware to use (MUXs, Decoders, etc....)

Q3)

Gate level:

- only focuses on using basic logic gates while designing

- uses the built-in Verilog gates just as AND, OR, NOT,

- not suitable for large designs (Processors, DSPs,

- synthesizable

Structural level:

- Only focuses on describing the hardware structure and how different

- Modules are connected together

- Uses module instantiation, internal connections

- Suitable for large designs

- Synthesizable

RTL (Register Transfer Level):

- Focuses on how data are transferred between registers and how it's processed by combinational logic

Uses always block

Behavioral Level:

Focuses on Describing how the hardware behaves

Uses always blocks

Suitable for describing complex algorithms which is hard to be described in a hardware connection

Part 3

Gate level Full adder

```
module GL_FullAdder (  
    input A, B, Cin,  
    output sum, Cout  
);  
  
    wire xor1, and1, and2;  
  
    xor (xor1, A, B);  
    and (and1, A, B);  
    and (and2, xor1, Cin);  
    xor (sum, xor1, Cin);  
    or (Cout, and2, and1);  
endmodule
```

Half Adder

```
module HalfAdder (  
    input A, B,  
    output sum, Cout  
);  
  
    xor (sum, A, B);  
    and (Cout, A, B);  
  
endmodule
```

Structural Level Full Adder

```
module SL_FullAdder (
    input A, B, Cin,
    output sum, Cout
);

    wire xor1, and1, and2;

    HalfAdder HA1 (
        .A(A),
        .B(B),
        .sum(xor1),
        .Cout(and1)
    );

    HalfAdder HA2 (
        .A(xor1),
        .B(Cin),
        .sum(sum),
        .Cout(and2)
    );

    assign Cout = and1 | and2;

endmodule
```

Behavioral Level Full Adder

```
module BL_FullAdder (
    input A, B, Cin,
    output sum, Cout
);

    always @(*) begin
        {Cout, sum} = A + B + Cin;
    end

endmodule
```

Testbench

```
module FullAdder_tb;

    reg A, B, Cin;
```

```

wire sum_BL, Cout_BL, sum_SL, Cout_SL, sum_GL, Cout_GL;
integer i;

BL_FullAdder dut_bl (
    .A(A),
    .B(B),
    .Cin(Cin),
    .sum(sum_BL),
    .Cout(Cout_BL)
);

SL_FullAdder dut_sl (
    .A(A),
    .B(B),
    .Cin(Cin),
    .sum(sum_SL),
    .Cout(Cout_SL)
);

GL_FullAdder dut_gl (
    .A(A),
    .B(B),
    .Cin(Cin),
    .sum(sum_GL),
    .Cout(Cout_GL)
);

initial begin
    for (i = 0; i < 8; i = i + 1) begin
        {Cin, B, A} = i;
        #10;

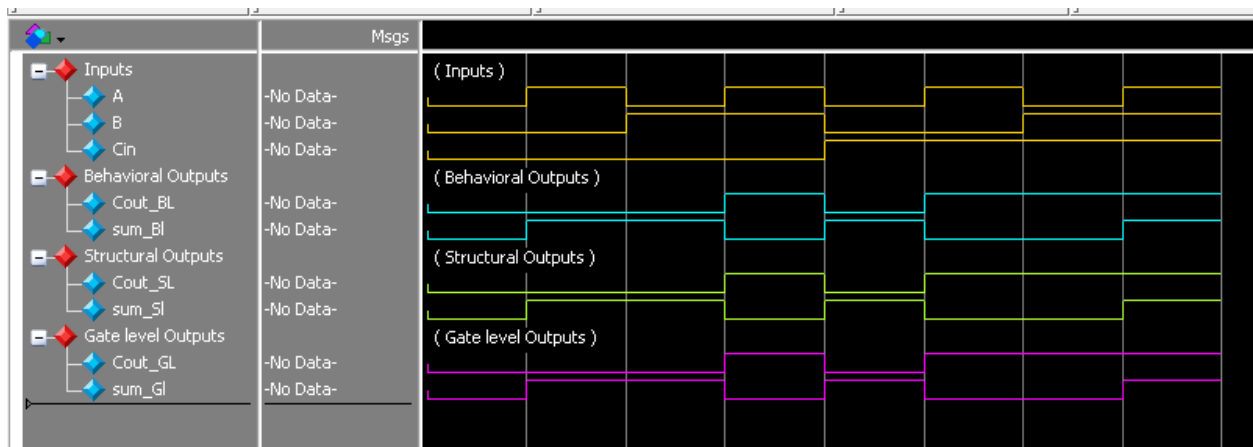
        // assign values to inputs from 000 till 111
        // so that all possibilities would be included
    end

    $stop;
end

endmodule

```

Questasim Waveform



- **All of the abstraction levels worked fine**